KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 1
붕어빵 틀과 붕어빵

틀 하나로 붕어빵을
여러 개 찍어 내기

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
```

오늘의 순서

1 왜 필요한가 4–6쪽

2 가장 작은 틀 7–9쪽

3 self와 __init__ 10–20쪽

4 메서드 21–24쪽

5 오류와 함정 25–28쪽

6 실습 29–31쪽

3쪽은 노트북 준비, 32–33쪽은 막힐 때와 한 장 요약이에요.

들어가기

23에서 이어서 해요

- ✓ 변수, 타입, 함수 type() 12
- ✓ 리스트, f-string, for 13
- ✓ 딕셔너리 15
- ✓ def, 매개변수, return 19
- ✓ 기본값, 지역 변수 20

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

1. 왜 필요한가

붕어빵을 딕셔너리로 담으면 값과 할 일이 따로 놀아요

[1]
```python
b1 = {"flavor": "팥", "price": 1000}
b2 = {"flavor": "슈크림", "price": 1200}

def intro(bread):
    print(bread["flavor"], "붕어빵", bread["price"], "원")

intro(b1)
intro(b2)
```

← 값은 딕셔너리에

← 할 일은 밖의 함수에

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

[2]
```python
b3 = {"flavr": "피자", "price": 1500}
intro(b3)
```

← 키를 틀려도 조용해요

```
KeyError: 'flavor'
```

← 쓸 때 터져요

값과 할 일을 한 틀에 묶어 두는 방법이 클래스예요.

1. 왜 필요한가

틀 하나로 붕어빵을 여러 개 찍어요

```
class Bread

맛     flavor = ?
가격   price  = ?

할 일   intro()  ← 메서드
```

틀 = 클래스

팥, 1000

슈크림, 1200

a

flavor	"팥"
price	1000

b

flavor	"슈크림"
price	1200

속성

붕어빵 = 객체

사실 int, str, list도 틀이에요

[3]
```python
print(type(3))
print(type("팥"))
print(type([1, 2]))
```

```
<class 'int'>
<class 'str'>
<class 'list'>
```

← int 틀로 찍은 객체
← str 틀로 찍은 객체
← list 틀로 찍은 객체

[4]
```python
xs = [1, 2]
xs.append(3)
print(xs)
print("Pat".lower())
```

← list 틀의 메서드 (13)
← str 틀의 메서드 (22)

```
[1, 2, 3]
pat
```

이미 써 온 틀처럼, 붕어빵 틀을 직접 만들어 봐요.

2. 가장 작은 틀

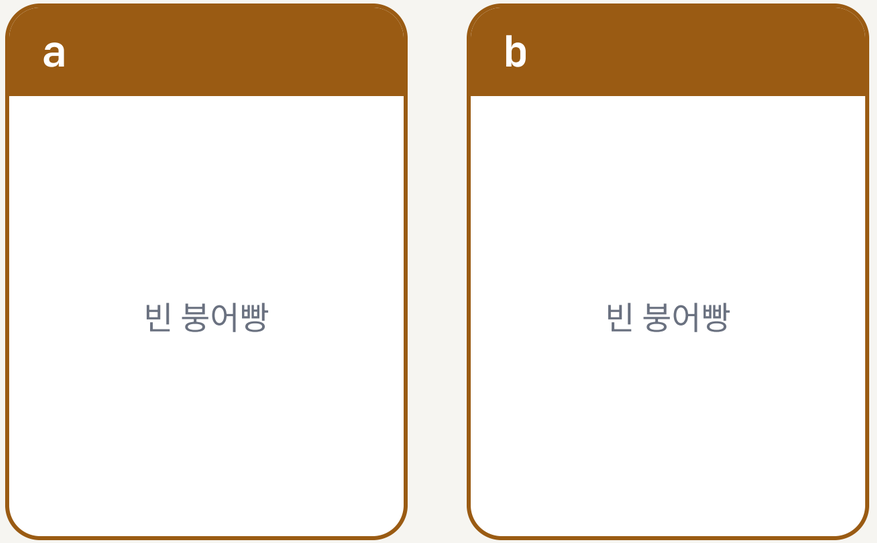
빈 틀을 만들고 붕어빵 두 개를 찍어요

[1]

```python
class Bread:        ← 틀 이름, 첫 글자는 대문자
    pass            ← 아직 빈 틀

a = Bread()
b = Bread()         ← 괄호를 붙이면 한 개 찍혀요
print(type(a))
```

```
<class '__main__.Bread'>   ← 이 노트북(__main__)의 Bread
```



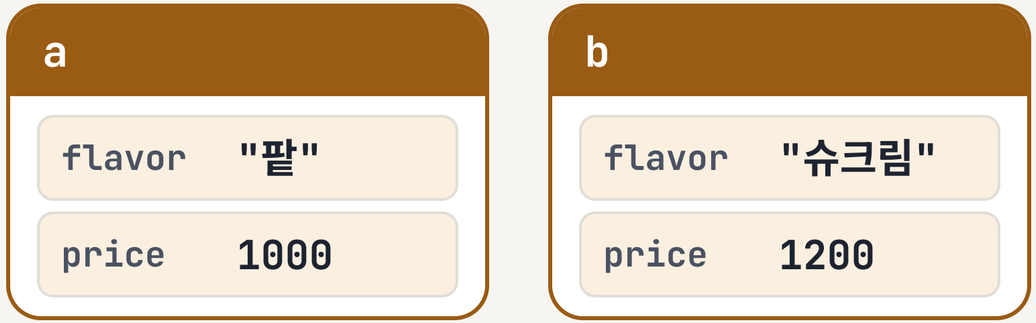
a와 b는 같은 틀로 찍은 서로 다른 붕어빵이에요.

2. 가장 작은 틀

점(.)으로 붕어빵에 값을 붙여요

[2]
```python
a.flavor = "팥"
a.price = 1000
b.flavor = "슈크림"
b.price = 1200
print(a.flavor, a.price)
print(b.flavor, b.price)
```

```
팥 1000
슈크림 1200
```



붕어빵에 붙은 값 = 속성

	딕셔너리 (15)	붕어빵
값 넣기	`b1["flavor"] = "팥"`	`a.flavor = "팥"`
값 꺼내기	`b1["flavor"]`	`a.flavor`

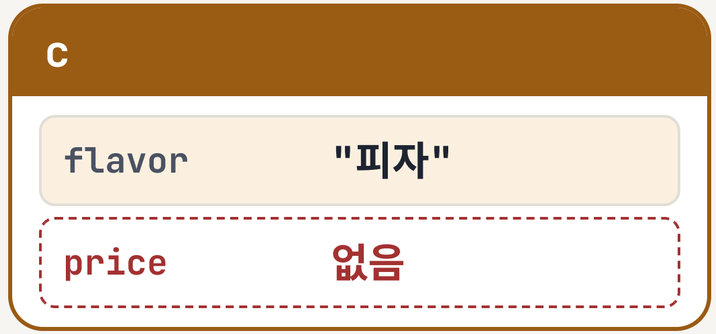
a.flavor는 "a의 flavor"로 읽어요.

2. 가장 작은 틀

손으로 붙이면 길고, 하나 빠뜨리기 쉬워요

```
[3]  c = Bread()
     c.flavor = "피자"    ← price 줄을 빠뜨렸어요
     print(c.price)
                                                    Python
```

```
AttributeError: 'Bread' object has no attribute 'price'
```



붕어빵이 10개라면 30줄을 손으로 적어요



한 줄만 빠져도 그 붕어빵은 칸이 비어요

찍는 순간 재료를 한 번에 넣게 하면 빠뜨릴 일이 없어요.

3. self와 __init__

먼저 19에서 배운 함수로 한 번에 붙여 봐요

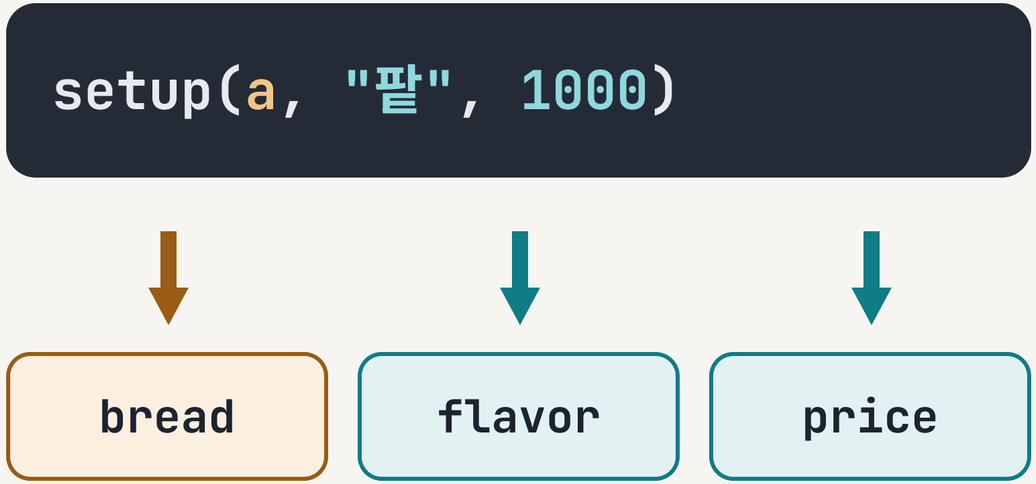
[1]
```python
class Bread:
    pass

def setup(bread, flavor, price):
    bread.flavor = flavor
    bread.price = price

a = Bread()
setup(a, "팥", 1000)
print(a.flavor, a.price)
```

Python

```
팥 1000
```



첫 칸이 어느 붕어빵에 붙일지 정해요

이 칸이 곧 self가 돼요

bread.flavor = flavor는 a에 맛 칸을 붙이는 것과 같아요.

3. self와 __init__

그 함수를 틀 안에 넣으면 __init__이에요

밖에 둔 함수

```python
def setup(bread, flavor, price):
    bread.flavor = flavor
    bread.price = price

a = Bread()
setup(a, "팥", 1000)
```

```
팥 1000
```

틀 안의 __init__

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price


a = Bread("팥", 1000)
```

```
팥 1000
```

① **들여쓰기** — class 아래로 넣어요

② **setup → __init__** — 찍을 때 저절로 불리는 약속된 이름

③ **bread → self** — 이름만 바뀌고 하는 일은 같아요

④ **Bread("팥", 1000)** — a를 넘기지 않아요. 파이썬이 넣어 줘요

3. self와 __init__

괄호 안의 재료가 __init__의 칸에 들어가요

```
a = Bread("팥", 1000)
```

파이썬이 넣어 줘요

괄호 안의 재료가 순서대로 (19의 매개변수)




```
def __init__(self, flavor, price):
```

칸은 셋, 재료는 둘. 첫 칸 self는 적지 않아요

__init__ = 앞뒤 밑줄 두 개씩, initialize(처음 준비)

3. self와 __init__

__init__은 찍는 순간 저절로 불려요

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        print("반죽 붓는 중:", flavor)
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
print(a.flavor, b.flavor)
```

← 확인용 print

← __init__을 직접 부르지 않았어요

```
반죽 붓는 중: 팥
반죽 붓는 중: 슈크림
팥 슈크림
```

← a를 찍을 때 한 번

← b를 찍을 때 한 번

붕어빵마다 꼭 있어야 하는 값을 찍는 순간 넣어 둘 때 써요.

3. self와 __init__

self는 지금 찍고 있는 그 붕어빵이에요

```
a = Bread("팥", 1000)
```

```
b = Bread("슈크림", 1200)
```

```
def __init__(self, flavor, price):
    self.flavor = flavor
    self.price = price
```



a를 찍는 동안 self.flavor = flavor는 a.flavor = "팥"과 같아요.

3. self와 __init__

self는 지금 찍고 있는 그 붕어빵이에요

```
a = Bread("팥", 1000)
```

```
b = Bread("슈크림", 1200)
```

```
def __init__(self, flavor, price):
    self.flavor = flavor
    self.price = price
```



첫째 줄에서 self는 a, 둘째 줄에서 self는 b예요. 코드 한 벌로 붕어빵마다 다른 값을 붙여요.

a = Bread("팥", 1000) 한 줄에서 일어나는 일

1. **`Bread("팥", 1000)`**
   빈 붕어빵이 하나 생겨요
2. **`__init__(빈 붕어빵, ...)`**
   빈 붕어빵이 self로 들어가요
3. **`self.flavor = "팥"`**
   맛 칸을 붙여요
4. **`self.price = 1000`**
   가격 칸을 붙여요
5. **`a = 다 된 붕어빵`**
   끝나면 a에 담아요



②~④에서 self는 이 붕어빵이에요. ⑤에서 a라는 이름이 붙어요.

3. self와 __init__

눌러 보기: 붕어빵 기계로 한 단계씩 찍어 봐요

맛 팥 슈크림 피자 고구마

가격 1000 1200 1500

틀 Bread

```
__init__(self, flavor,
         price)
```

찍으면 여기서 붕어빵이 만들어져요

```
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
```

한 단계씩 ▶ 한 번에 찍기 처음부터

찍은 붕어빵

아직 없어요. 찍으면 여기에 쌓여요.

맛과 가격을 고르고 **한 단계씩**이나 **한 번에 찍기**를 눌러요. 브라우저로 슬라이드를 열면 눌러져요.

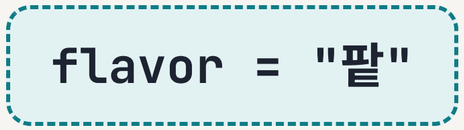
3. self와 __init__

self.flavor = flavor는 왼쪽과 오른쪽이 달라요

```
self.flavor = flavor
```



붕어빵에 붙는 칸. 계속 남아서 a.flavor로 꺼내요



들고 온 재료. __init__이 끝나면 사라져요 (20의 지역 변수)

이름이 같아서 헷갈리면 self.이 붙었는지만 봐요.

3. self와 __init__

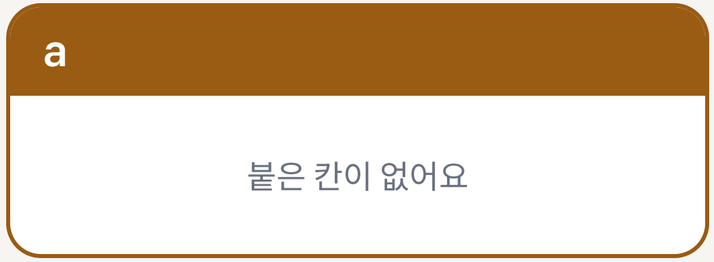
self.를 빼면 재료가 붕어빵에 안 붙어요

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        flavor = flavor
        price = price

a = Bread("팥", 1000)
print(a.flavor)
```

← 자기 자신에 다시 넣었을 뿐

```
AttributeError: 'Bread' object has no attribute 'flavor'
```



× ~~flavor = flavor~~ → ✓ `self.flavor = flavor`

붕어빵에 남길 값은 꼭 self.이름 = 값으로 적어요.

3. self와 __init__

재료로 받지 않는 칸도 만들 수 있어요

[5]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price
        self.baked = False   ← 괄호로 받지 않고 처음부터 False

a = Bread("팥", 1000)
print(a.flavor, a.price, a.baked)
```

```
팥 1000 False
```

a

flavor	"팥"
price	1000
baked	False

도장 0개, 판매 0개처럼 모두 같은 값으로 시작하는 칸에 써요.

4. 메서드

틀 안의 함수가 메서드예요

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.intro()
b.intro()
```

← 메서드. 첫 칸은 self

← 그 붕어빵의 맛과 가격

← 붕어빵에게 시켜요

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

intro(b1)처럼 넘기지 않고 a.intro()로 시켜요.

a.intro()는 Bread.intro(a)와 같아요

```
a.intro()
```

→

```
Bread.intro(a)
```

점 앞의 a가 self 칸으로 들어가요

[2]
```
Bread.intro(a)
Bread.intro(b)
```
Python

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
```

← a.intro(), b.intro()와 똑같아요

4. 메서드

메서드로 값을 바꾸고, return으로 돌려받아요

[3]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def discount(self, won):
        self.price = self.price - won

    def total(self, qty):
        return self.price * qty

a = Bread("팥", 1000)
b = Bread("슈크림", 1200)
a.discount(200)
print(a.price, b.price)
print(a.total(3))
```

← 값 바꾸기

← 돌려주기

```
800 1200
2400
```

a	
flavor	"팥"
price	800

b	
flavor	"슈크림"
price	1200

a.discount(200)의 self는 a. b는 그대로예요

4. 메서드

클래스는 재료를 빠뜨리면 찍을 때 바로 알려 줘요

	딕셔너리와 함수 (1파트)	클래스
모양	`b1 = {"flavor": "팥", ...}`	`a = Bread("팥", 1000)`
할 일	`intro(b1)`	`a.intro()`
할 일은 어디에	함수가 밖에 따로	틀 안에 같이
값 꺼내기	`b1["flavor"]`	`a.flavor`
재료를 빠뜨리면	쓸 때 KeyError	찍을 때 바로 TypeError
어울리는 곳	잠깐 담아 두는 값	같은 모양을 여러 개 찍을 때

5. 오류와 함정

재료 수가 모자라면 missing argument

[1]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro():
        print("붕어빵")

a = Bread("팥")   ← 재료가 하나뿐
```

```
TypeError: Bread.__init__() missing 1 required positional argument: 'price'
```

읽는 법: __init__에 꼭 필요한 칸 price가 1개 모자라요

× ~~a = Bread("팥")~~ → ✓ `a = Bread("팥", 1000)`

self는 파이썬이 채우니까, self를 뺀 칸 수만큼 재료를 넣어요.

5. 오류와 함정

메서드에 self를 빼면 takes 0 positional arguments

앞 셀의 틀 일부

```
    def intro():
        print("붕어빵")
```

[2]

```
a = Bread("팥", 1000)
a.intro()
```

Python

```
TypeError: Bread.intro() takes 0 positional arguments but 1 was given
```

읽는 법: 받을 칸은 0개인데 점 앞의 a가 1개 들어왔어요

× ~~`def intro():`~~ → ✓ `def intro(self):`

a.intro()는 Bread.intro(a)라서 a가 하나 들어가요.

5. 오류와 함정

밑줄을 하나만 적으면 takes no arguments

[3]
```python
class Bread:
    def _init_(self, flavor, price):   ← 밑줄이 하나씩
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
```

Python

```
TypeError: Bread() takes no arguments
```

읽는 법: 약속된 이름 __init__을 못 찾아서, 재료를 받지 않는 틀이 됐어요

× ~~`def _init_(self, ...)`~~ → ✓ `def __init__(self, ...)`

__int__처럼 글자가 빠져도 같은 오류가 나요.

5. 오류와 함정

붕어빵을 print하면 이상한 글자가 나와요

[4]
```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

a = Bread("팥", 1000)
print(a)
```

```
<__main__.Bread object at 0x7f440c2dbb60>
```
← 오류는 아니에요

```
<__main__.Bread object at 0x7f440c2dbb60>
```

- 이 노트북의 Bread 틀의
- 객체(붕어빵)
- 컴퓨터 안의 자리. 실행할 때마다 달라요

지금은 a.flavor처럼 칸을 꺼내 찍고, 25의 __str__로 보기 좋게 바꿔요.

6. 실습

붕어빵 가게: 주문 받은 붕어빵을 소개하고 합계 내기

[1]

```python
class Bread:
    def __init__(self, flavor, price):
        self.flavor = flavor
        self.price = price

    def intro(self):
        print(self.flavor, "붕어빵", self.price, "원")

order = [Bread("팥", 1000), Bread("슈크림", 1200), Bread("팥", 1000)]
for b in order:
    b.intro()
total = sum([b.price for b in order])
print("합계", total, "원")
```

Python

← 객체도 리스트에

← 하나씩 소개

← 23의 컴프리헨션

```
팥 붕어빵 1000 원
슈크림 붕어빵 1200 원
팥 붕어빵 1000 원
합계 3200 원
```

← b.price만 모아 더했어요

6. 실습

카페 음료: 기본값 사이즈와 메뉴 이름표 만들기

[2]
```python
class Drink:
    def __init__(self, name, price, size="M"):   ← 20의 기본값
        self.name = name
        self.price = price
        self.size = size

    def label(self):   ← 글자를 돌려줘요
        return f"{self.name}({self.size}) {self.price:,}원"

d1 = Drink("latte", 5000)
d2 = Drink("tea", 4000, "L")
print(d1.label())
print(d2.label())
```

Python

```
latte(M) 5,000원
tea(L) 4,000원
```

d1

name	"latte"
price	5000
size	"M"

d2

name	"tea"
price	4000
size	"L"

size를 안 넣으면 "M"

6. 실습

도장 카드: 처음 0개에서 손님마다 따로 쌓기

[3]

```python
class Card:
    def __init__(self, name):
        self.name = name
        self.stamps = 0                    ← 처음 값 0

    def add(self):
        self.stamps = self.stamps + 1      ← 자기 도장 +1

kim = Card("kim")
lee = Card("lee")
kim.add()
kim.add()
kim.add()
lee.add()
print(kim.name, kim.stamps)
print(lee.name, lee.stamps)
```

Python

```
kim 3
lee 1
```

kim

name	"kim"
stamps	3

lee

name	"lee"
stamps	1

같은 틀, 카드마다 칸이 따로

정리

막히면 오류의 마지막 줄을 봐요

`TypeError: ... missing 1 required positional argument` 찍을 때 재료가 모자라요	✓ self 뒤 칸 수만큼
`TypeError: ... takes 0 positional arguments but 1 was given` 메서드에 self가 없어요	✓ `def intro(self):`
`TypeError: Bread() takes no arguments` __init__ 이름이 틀렸어요	✓ 밑줄 두 개씩
`AttributeError: 'Bread' object has no attribute ...` 칸 이름 오타이거나 self.를 뺐어요	✓ self.이름 = 값
`NameError: name 'flavor' is not defined` 메서드 안에서 flavor만 썼어요	✓ `self.flavor`
`<__main__.Bread object at 0x...>` 붕어빵을 통째로 print했어요	✓ a.flavor, 25의 __str__
`TypeError: ... missing 1 required positional argument: 'self'` a = Bread처럼 괄호 없이 담았어요	✓ Bread(재료)

헷갈리면 self 자리에 붕어빵 이름(a)을 넣어 소리 내어 읽어 봐요.

한 장 요약

`class Bread:`

틀 만들기

`self`

지금 그 붕어빵

`Bread("팥", 1000)`

붕어빵 찍기

`def intro(self):`

메서드 만들기

`def __init__(self, ...):`

찍을 때 재료 넣기

`a.intro()`

Bread.intro(a)와 같아요

`self.flavor = flavor`

붕어빵에 칸 붙이기

`a.flavor`

속성 꺼내기

self는 "방금 찍은 붕어빵", "메서드를 부른 붕어빵"이에요.

다음: 25에서 __str__, __repr__로 붕어빵을 보기 좋게 찍어요.